WEEK 4 · LESSON 21 OF 24 · HANDOUT

Attention and Transformers

Study time: **about 55 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=eMlx5fFNoYc — open the lesson online to watch inside the course.

🎓 **Go deeper:** Andrej Karpathy: Let's build GPT from scratch, in code — https://www.youtube.com/watch?v=kCc8FmEb1nY

🎓 **Go deeper:** MIT 6.S191 (2024): Recurrent Neural Networks, Transformers and Attention — https://www.youtube.com/watch?v=dqoEU9Ac3ek

🎓 **Go deeper:** Andrej Karpathy: Intro to Large Language Models (1-hour talk) — https://www.youtube.com/watch?v=zjkBMFhNj_g

Your goal in this lesson: Compute self-attention step by step, explain queries, keys, values, heads and positional encodings, and build a small transformer classifier in PyTorch.

The idea that changed AI

In 2017 a Google paper called **"Attention Is All You Need"** introduced the **transformer**. Instead of reading words one at a time like an RNN, every word looks at **every other word at once** and decides which ones matter for its meaning. In "the bank of the river", the word "bank" pays attention to "river" and so means riverside, not money. Transformers power ChatGPT, Claude, Gemini, translation, and modern vision and speech models.

Self-attention in four steps

Each token's embedding is turned into three vectors by three learned weight matrices:

- **Query (Q)** — "what am I looking for?"
- **Key (K)** — "what do I contain?"
- **Value (V)** — "what information do I pass on?"

Then:

1. **Scores:** compare every query with every key (dot product).
2. **Scale:** divide by √(dimension) to keep numbers moderate.
3. **Softmax:** turn each row of scores into attention weights that sum to 1.
4. **Mix:** each token's new vector is the weighted average of all the values.

> $Attention(Q, K, V) = softmax(Q\,K^T \div \sqrt{d})\,V$

```
import torch
import torch.nn as nn
import torch.nn.functional as F

torch.manual_seed(0)
tokens = ["money", "sent", "to", "landlord"]
d = 8
x = torch.randn(len(tokens), d)                    # pretend embeddings
Wq, Wk, Wv = (torch.randn(d, d) * 0.5 for _ in range(3))

Q, K, V = x @ Wq, x @ Wk, x @ Wv
scores = Q @ K.T / d ** 0.5                        # every token vs every token
weights = F.softmax(scores, dim=-1)                # each row sums to 1
output = weights @ V

print("attention weights (row = token looking, column = token looked at):")
for tok, row in zip(tokens, weights):
    print(f"{tok:9s}", [round(w, 2) for w in row.tolist()])
print(output.shape)
```

With random weights the pattern is meaningless; training makes it useful. The output has the same shape as the input — one new, **context-aware** vector per token.

PyTorch has this built in, optimised for GPUs:

```
same = F.scaled_dot_product_attention(Q.unsqueeze(0), K.unsqueeze(0), V.unsqueeze(0))
print(torch.allclose(same[0], output, atol=1e-5))
```

Multi-head attention

One attention pattern cannot capture everything (grammar, meaning, references…). **Multi-head attention** runs several attention operations ("heads") in parallel on smaller slices of the vectors and joins the results. `nn.MultiheadAttention(embed_dim=32, num_heads=4)` gives four heads of 8 dimensions each.

Order: positional encodings

Attention by itself treats a sentence like a bag of words — it has no idea which word came first. Transformers **add a position vector** to each token's embedding (fixed sine waves, or learned position embeddings), so "dog bites man" and "man bites dog" differ.

The transformer block

> *x → multi-head self-attention → add & normalise → feed-forward network → add & normalise*

The "add" steps are residual connections (Lesson 10), and "normalise" is **layer normalisation**. Models stack many blocks: about 12 in early BERT, around a hundred in today's largest language models.

- **Encoder** blocks (BERT style) see the whole text — good for classification and search.
- **Decoder** blocks (GPT style) only look at earlier tokens (a **causal mask**) and are trained to predict the next token — that is how chatbots generate text.

A small transformer classifier

Let's give the customer-message router from Lesson 19 a transformer encoder.

```
import re
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F

TEMPLATES = {
    "payment": ["i sent money to {who} but it did not arrive", "my payment to {who} failed",
                "money was deducted twice for {who}", "how do i reverse a payment to {who}",
                "the merchant {who} did not receive my payment"],
    "network": ["my {what} is not working in {where}", "no {what} signal in {where} since
morning",
                "{what} is very slow in {where}", "i cannot connect to {what} in {where}",
                "the {what} keeps dropping in {where}"],
    "account": ["i forgot my {thing}", "please reset my {thing}", "my {thing} is blocked",
                "how do i change my {thing}", "someone asked for my {thing} on the phone"],
}
FILL = {"who": ["my brother", "the shop", "a wrong number", "the school", "my landlord", "the
supermarket"],
        "what": ["internet", "data bundle", "4g", "network", "airtime bundle"],
        "where": ["kigali", "huye", "musanze", "rubavu", "nyagatare", "my village"],
        "thing": ["pin", "password", "sim card", "mobile money pin", "account"]}
TOPICS = list(TEMPLATES)

def make_messages(n, seed):
    rng = np.random.default_rng(seed)
    texts, labels = [], []
    for _ in range(n):
        label = int(rng.integers(0, 3))
        text = re.sub(r"\{(\w+)\}", lambda m: str(rng.choice(FILL[m.group(1)])),
                      str(rng.choice(TEMPLATES[TOPICS[label]])))
        if rng.random() < 0.3:
            text = str(rng.choice(["hello ", "please ", "good morning ", "help "])) + text
        if rng.random() < 0.3:
            text += str(rng.choice([" thank you", " urgent", " please help", " today"]))
        texts.append(text)
        labels.append(label)
    return texts, torch.tensor(labels)

train_texts, y_train = make_messages(800, seed=0)
test_texts, y_test = make_messages(200, seed=1)
for t, l in zip(train_texts[:4], y_train[:4]):
    print(f"[{TOPICS[l]}] {t}")
```

```
vocab = {"<pad>": 0, "<unk>": 1}
for text in train_texts:                      # build the vocabulary from training text only
    for word in text.split():
        vocab.setdefault(word, len(vocab))

def encode(text, max_len=14):
    ids = [vocab.get(w, vocab["<unk>"]) for w in text.split()][:max_len]
    return ids + [vocab["<pad>"]] * (max_len - len(ids))

X_train = torch.tensor([encode(t) for t in train_texts])
X_test = torch.tensor([encode(t) for t in test_texts])

def fit(model, epochs=30, lr=1e-2):
    opt = torch.optim.Adam(model.parameters(), lr=lr)
    for epoch in range(epochs):
        model.train()
        perm = torch.randperm(len(X_train))
        for start in range(0, len(perm), 32):
            idx = perm[start:start + 32]
            opt.zero_grad()
            F.cross_entropy(model(X_train[idx]), y_train[idx]).backward()
            opt.step()
    model.eval()
    with torch.no_grad():
        return round((model(X_test).argmax(1) == y_test).float().mean().item(), 3)
print("vocabulary size:", len(vocab), "| first message as ids:", X_train[0].tolist())
```

```
class TinyTransformer(nn.Module):
    def __init__(self, vocab_size, d=32, heads=4, layers=2, max_len=14, n_classes=3):
        super().__init__()
        self.tok = nn.Embedding(vocab_size, d, padding_idx=0)
        self.pos = nn.Embedding(max_len, d)                    # learned positions
        block = nn.TransformerEncoderLayer(d_model=d, nhead=heads, dim_feedforward=64,
                                           dropout=0.1, batch_first=True)
        self.encoder = nn.TransformerEncoder(block, num_layers=layers,
enable_nested_tensor=False)
        self.out = nn.Linear(d, n_classes)

    def forward(self, x):
        positions = torch.arange(x.shape[1], device=x.device)
        h = self.tok(x) + self.pos(positions)
        h = self.encoder(h, src_key_padding_mask=(x == 0))     # ignore padding
        mask = (x != 0).unsqueeze(-1).float()
        return self.out((h * mask).sum(1) / mask.sum(1))

torch.manual_seed(0)
tiny = TinyTransformer(len(vocab))
print("parameters:", sum(p.numel() for p in tiny.parameters()))
print("test accuracy:", fit(tiny, epochs=20, lr=1e-3))
```

The transformer also routes the messages almost perfectly. On this easy, template-made data, the simple bag of embeddings was already enough — the transformer's advantage shows on long, real text where word order and context matter, and above all when it is **pretrained** on huge amounts of text and then fine-tuned (transfer learning again, as in Lesson 16).

Why transformers won

- **Parallel:** all tokens are processed at once, so GPUs train them much faster than RNNs.
- **Long range:** any word can attend directly to any other, however far apart.

- **Scale:** performance keeps improving with more data, parameters and computing — the path to large language models.

The price: attention compares every token with every other, so cost grows with the **square** of the text length.

Key terms

- **Self-attention** — every token weighs every other token to build a context-aware vector.
- **Query, key, value** — the three learned projections used by attention.
- **Multi-head attention** — several attention patterns in parallel.
- **Positional encoding** — information about word order added to embeddings.
- **Transformer block** — attention + feed-forward, with residuals and layer norm.
- **Encoder / decoder** — sees all tokens / sees only earlier tokens (for generation).

Remember this: Attention lets every token look at every other token at once; stacked with feed-forward layers, residuals and positions, it forms the transformer behind modern AI.

Name: _____ Date: _____

Activity

Task 1

In the attention example, multiply `Wq` by 10. What happens to the attention weights, and why does dividing by √d matter?

Task 2

Add a causal mask (`torch.triu`) to the scores so tokens cannot look at later tokens. Print the new weights.

Task 3

Train `TinyTransformer` with 1 layer and 1 head. Does it still work on these messages?

Task 4

Count how many attention scores a 2,000-token document needs per head and per layer.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What does self-attention let each token do?
   - A. Sort the sentence
   - B. Ignore the other tokens
   - C. Delete padding
   - D. Weigh every other token to build a context-aware representation
2. In attention, what are the scores computed from?
   - A. Dot products of queries and keys
   - B. Values and labels
   - C. The learning rate
   - D. Pixel colours
3. Why do transformers need positional encodings?
   - A. To make the model smaller
   - B. To replace embeddings
   - C. Attention alone ignores word order
   - D. To mask padding
4. What is multi-head attention?
   - A. A type of pooling
   - B. Two GPUs
   - C. Several attention patterns computed in parallel
   - D. A network with many outputs
5. How do GPT-style decoders generate text?
   - A. By sorting words
   - B. By averaging embeddings
   - C. By looking only at earlier tokens and predicting the next one
   - D. By reading images
6. What is a main cost of attention?
   - A. It cannot use GPUs
   - B. It forgets everything
   - C. Its cost grows with the square of the sequence length
   - D. It needs no data

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.